Correct Optimizations: Phase 2 Report Based on "Minotaur: A SIMD-Oriented Synthesizing Superoptimizer" [1]

Kilian Schulz
Technical University Munich

Ilja Gamza
Technical University Munich

Tomás Gaspar
Technical University Munich

1 Introduction

Minotaur [1] is a superoptimizer that after computing optimizations caches those in a data base. These optimizations consist of a key, which is the original code, and a value, which is an equivalent rewrite of the original code. This caching is done in order to reduce the already substantial compile times of Minotaur. Even when using this caching mechanism compile times are very long, up to hundreds of time longer than without Minotaur.

Currently Minotaur directly uses its extracted code segments as keys. This approach can easily lead to near misses where for example an equivalent pice of code is already cached but has a different order of arguments. To mitigate this we extended Minotaur [1] to support key canonicalization using multiple different steps.

The core challenge is to keep the semantics of the keys, as changing those would lead to incorrect optimizations, and to enable application of the rewrites which fit to the original key. This requires both a forward and backwards conversion from and to the canonical from for both the key and the value (the rewrite). At the same time we try to reduce the canonical as much as possible in order to map as many semantically equivalent keys to one canonical key.

2 Research Proposal

3 Contributions

Our code is available on GitHub [1].

[1] Extended Minotaur Version

4 Evaluation

4.1 Setup

4.2 Results

The key metrics which measured for the following benchmarks were the cache size, cache hit rate and number of applied rewrites.

Since our canonicalization steps are compositional we measure the impact of each step and combination of steps.

We also tested changing the order of canonicalization steps where possible and measured the effect.

We also measure the total compilation time for several select software projects.

5 Discussion

6 Conclusion

References

[1] Zhengyang Liu, Stefan Mada, and John Regehr. "Minotaur: A SIMD-Oriented Synthesizing Superoptimizer". In: *Proceedings of the ACM on Programming Languages (OOPSLA2)* 8.OOPSLA2 (Oct. 2024). https://doi.org/10.1145/3689766, pp. 1–25. DOI: 10.1145/3689766.